Ders 11 Genişletilmiş Notu: Large Class, Primitive Obsession ve Feature Envy Kod Kusurları

Ders 11 Genişletilmiş Notu: Large Class, Primitive Obsession ve Feature Envy Kod Kusurları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Yeri ve Başlangıç Noktası
- 2. Large Class (Aşırı Büyük Sınıf)
- 3. Extract Class (Sınıf Çıkartma)
- 4. Extract Subclass (Alt Sınıf Çıkartma)
- 5. Duplicate Observed Data (Gözlenen Verinin Çoğullanması)
- 6. Divergent Change ve Shotgun Surgery
- 7. Move Method (Metot Taşınması)
- 8. Primitive Obsession (İlkel Tiplerin Aşırı Kullanımı)
- 9. Replace Data Value with Object
- 10. Replace Type Code with Class
- 11. Replace Type Code with Subclasses
- 12. Replace Type Code with State/Strategy
- 13. Replace Array with Object
- 14. Feature Envy (Özellik Kıskançlığı)
- 15. Kapanış: Kalıplar ve Kod Kusurları
- Örnek Sorular
- Dersin Toparlaması

1. Dersin Yeri ve Başlangıç Noktası

Bu ders, önceki mevcut kayıtta (ders 9) başlayan kod kusurları (code smells) ve düzenleme eylemleri (refactoring) dizisinin devamıdır. Hoca slayt dosyasını açıp önceki derste nerede kalındığına baktı. Son işlenen slayt 30'daki "Replace Method with Method Object" düzenlemesiydi (ekrandan, 00:02:10). Hoca bu noktanın slayt sayısı bakımından dosyanın yaklaşık yarısı olduğunu, ama kod kusurları ve refactoring konusunun sonuna yaklaşıldığını söyledi.

Hocanın planına göre bu derste kalan kod kusurları ve refactoring eylemleri bitirilecek, bütüncül refactoring örneği ise en sona bırakılacaktır. Ders yaklaşık bir ders saati sürdü. Bugünkü ilk konu slayt 31'deki Large Class'tır (ekrandan, 00:10:00).

2. Large Class (Aşırı Büyük Sınıf)

2.1 Adlandırma ve kök neden

Hoca "Large Class" adını "aşırı büyük sınıf" diye çevirdi. Gerekçesi şudur: bir sınıfın büyük olması tek başına bir sorun değildir; sorun büyüklüğün aşırı olmasıdır. Bu yüzden "aşırı" sözcüğünü özellikle vurgulamak istedi.

Aşırı büyük sınıftaki en önemli sorun ve bu sorunun kök nedeni, sınıfın sorumluluklarının aşırı fazla olmasıdır. Sorumlulukların sayısı arttıkça bunların birbiriyle ilgi oranı düşer. İlgi ora-

nının düşmesi de uyumun (cohesion) düşmesine yol açar. Slayt aynı fikri şöyle verir: “Sınıfın büyük olması, sınıfa ilgisiz sorumluluklar atandığının göstergesidir. Böylece uyum düşecektir.” (ekrandan, 00:10:00).

Hoca burada dersin ilk haftalarında konuşulan iki temel kalite ölçüsünü hatırlattı: bağlaşım (coupling) ve uyum (cohesion). Bağlaşım, nesnelerin birbirleriyle ilişki içinde bulunma oranı üzerinden konuşulmuştu. Uyum ise bir sınıfın içindeki öğelerin birbirine ne kadar yakın ve ne kadar ilgili olduğuyla tanımlanmıştı.

2.2 Büyüklüğün ölçüsü

Aşırı büyük sınıfları incelerken akla gelen ilk ölçü büyüklüktür, yani kod satırı sayısı (line of code). Hoca bunun yalnız satır sayısı olarak düşünülmemesini istedi. Üye alan sayısının fazlalığı da sınıfın büyüklüğünün bir göstergesidir (slaytta da ayrı madde olarak yer alır).

Nedeni bir kısır döngüdür. Sınıfa fazla sorumluluk atanmışsa, bu sorumlulukları yerine getirmek için gereken veriler de sınıfa konur. Sonra “madem bu kadar veri burada, şu işi de bu sınıf yapsın” denir. Böylece sınıf, hocanın deyişiyle bir kartopu gibi büyüyen bir döngüye girer.

Oysa nesneye dayalı yaklaşım “böl, parçala, yönet” stratejisini önerir. Hoca, daha önce verdiği iyi tasarım tanımını da hatırlattı: problem alanındaki varlıklar kaç parçaya ayrılıyorsa, tasarım da en az o düzeyde ya da daha ince alt düzeylere ayrılabilmelidir. Large Class bunun tam tersidir; aşırı büyük parçalardan oluşan yapılardır. Hoca, bu duruma “neşter atmak” gerektiğini söyledi.

2.3 Çözüme götüren düzenlemeler

Slayt 31 üç düzenleme sayar (ekrandan, 00:10:00):

- **Extract Class:** Birlikte anlam ifade eden üye alanlardan bir ya da daha fazla yeni sınıf oluşturulur.
- **Extract Subclass:** Oluşturulacak yeni sınıfın mevcut sınıfın alt sınıfı olması daha uygun olabilir. Örneğin mevcut sınıf bazı üyelerini her zaman kullanmıyorsa bunlar yeni bir alt sınıfa aktarılabilir.
- **Duplicate Observed Data:** İş mantığı ile kullanıcı arayüzü arasında yüksek bağımlılık varsa GUI sınıfları çok büyür. Bu durumda veri yeni bir uygulama alanı sınıfına kopyalanır ve aradaki eşgüdüm Observer tasarım kalıbına göre sağlanır.

Hoca Extract Class’ı Extract Method’a benzetti: orada metottan yeni bir metot çıkarılıyordu, burada sınıftan yeni bir sınıf çıkarılır. Yeni sınıfa yalnız üye alanlar değil, onlarla çalışan metotlar da taşınır. Bu iş tek bir sınıf çıkarmakla sınırlı değildir; kusurlu sınıf ne kadar büyükse ondan o kadar çok sınıf çıkarma olasılığı artar.

Çıkarılan sınıfın mevcut sınıfın alt sınıfı olması uygunsa yapılan iş Extract Subclass olur. Bunun ipucu, sınıfın metotlarında görülen ayrık erişim desenleridir. Metotlar üyeleri yeterli yoğunlukta kullanıyor olabilir; ama if-else yapılarında ya da üye alanların ve parametrelerin değerlerine göre “şu koşul gerçekleşmişse şu üye alanları kullan, öbür türlü bu üye alanları kullan” gibi bir desen ortaya çıkıyorsa alt sınıf düşünülmelidir.

Hocanın sözlü örneği maaş hesabıdır. Her çalışanın maaşı hesaplanır. Çalışan bir yöneticiyse maaşına, varsa ikramiyesi de eklenir. Kod her çalışan için “bu bir yönetici mi, öyleyse bonusunu da ekle” diyorsa ikramiye üyesi bazı durumlarda kullanılıyor, bazılarında kullanılmıyor demektir. Bu durumda yönetici sınıfını çalışan sınıfının alt sınıfı olarak çıkarmak daha doğrudur: her yönetici aynı zamanda bir çalışandır, ama ek olarak ikramiyeye sahiptir.

Hoca bu iki düzenlemenin genelde çözüme götürdüğünü, götürmediği durumlarda başka düzenlemelerin devreye gireceğini söyledi. Büyümenin üçüncü nedeni ise iş mantığı katmanı ile sunum katmanının birbirine karışmasıdır. Hocanın benzetmesiyle bu katmanlar ayrı değil, tek bir “ip yumağı” haline gelmiştir. İş mantığını yürüten sınıflar ile kullanıcı arayüzünü yürüten sınıflar arasında fazla bağımlılık (bağlaşım) varsa, arayüzü kodlayan sınıflar ister istemez çok büyür. Bu durumda kullanılacak düzenleme Duplicate Observed Data’dır.

3. Extract Class (Sınıf Çıkartma)

3.1 Tanım

Slayt 32'ye göre (ekrandan, 00:11:45):

- Birbiriyle ilgisiz iki iş yapan bir sınıfı, bu işlere göre ikiye ayırmaktır.
- İkinci iş ile ilgili üyelerin seçilerek yeni bir sınıfa taşınmasına dayanır.
- Bu yapılandırma birçok kod kusurunu düzeltir.

Hoca, Extract Class'ın düzelttiği ilk kusurun elbette Large Class olduğunu, ama Long Method'u da düzeltebileceğini söyledi. Uzun bir metodun içinde yapılan işler incelendiğinde, bunların diğer metotlarda yapılan işlerden farklı alanlarda çalıştığı görülebilir. Extract Method anlatılırken çıkarılan metot için "uygun bir yer" bulunması gerektiği söylenmişti. O uygun yer yepyeni bir sınıfsa, yapılan iş Extract Class olur; aradaki tek fark budur.

3.2 Person ve TelephoneNumber örneği

Slayttaki diyagram (ekrandan, 00:11:45):

```
ÖNCE:
  Person
    name
    officeAreaCode
    officeNumber
    getTelephoneNumber

SONRA:
  Person                            Telephone Number
    name           --officeTelephone-->  (1)
    getTelephoneNumber                    areaCode
                                          number
                                          getTelephoneNumber
```

`Person` sınıfında kişinin diğer bilgilerinin yanında iletişim bilgileri de vardır: adı, ofis telefonunun alan kodu ve ofis telefonunun numarası. Hoca, büyük olasılıkla bunların yanına cep telefonunun alan kodu ve numarası gibi benzer bilgilerin de geleceğini söyledi. Bu bilgiler geldikçe onlarla ilgili metotlar da gelir ve sınıf aşırı biçimde büyür.

Bunun yerine telefon numarasıyla ilgili veriler için yeni bir sınıf (`Telephone Number`) oluşturulur, bu bilgiler oraya taşınır ve `Person` sınıfından bu yeni sınıfa bir sahiplik ilişkisi kurulur. Slayttaki ilişkinin adı `officeTelephone`, çokluğu 1'dir. Cep telefonu eklemek gerekince aynı türden ikinci bir üye tanımlanır: biri ofis, biri cep için iki `TelephoneNumber` üyesi.

Hoca ayrı sınıfın başka bir kazancını da anlattı. Telefon numaralarıyla ilgili başka işlemler de yapılabilir: örneğin numaralardan yola çıkarak hangi telekom operatörüne ne kadar masraf düştüğü, hangi operatörde ne oranda abone olunduğu gibi analizler. Bu veri ayrı bir sınıfta durduğunda bu işler çok daha rahat yapılır.

Genel ilke şudur: kendi içinde sıkı ilişkili, ama sınıfın diğer üyeleriyle gevşek ilişkili bir içerik görülüyorsa, bu içeriği kendi içinde sıkı ilişkili ayrı sınıflara taşımak her bakımdan avantaj sağlar.

4. Extract Subclass (Alt Sınıf Çıkartma)

4.1 Tanım

Slayt 33: "Bir sınıfın bazı üyeleri her zaman kullanılmıyorsa, bu üyelerden yeni bir alt sınıf oluşturmak daha doğru olacaktır." (ekrandan, 00:14:30).

4.2 Job Item ve Labor Item örneği

```
ÖNCE:
  Job Item
```

```
    getTotalPrice
    getUnitPrice
    getEmployee

SONRA:
  Job Item
    getTotalPrice
    getUnitPrice
       ^ (kalıtım)
  Labor Item
    getUnitPrice
    getEmployee
```

(ekrandan, 00:14:30)

`Job Item` sınıfında yapılan işlerin toplam fiyatı, birim fiyatı ve işi kimin yaptığıyla ilgili üyeler vardır. Sistemdeki işlere bakıldığında bazı işlerin özellikle o işin ustası olan bir kişiye atandığı, bazı işlerin ise herhangi biri tarafından yapılabilir diye işaretlenip kimin yaptığının saklanmadığı görülür. Yani bazı durumlarda işi kimin yaptığıyla ilgilenilmez, bazı durumlarda ilgilenilir ve işlemler ona göre değişir.

Farklılaşan şey, bazı işlerin belirli bir çalışan tarafından yapılmasıdır. Bu yüzden `Job Item`'dan yeni bir alt sınıf oluşturulur: `Labor Item`. Hoca adı şöyle açıkladı: "job" da "labor" da iş demektir, ama "labor" daha çok emeğe, belirli bir kişinin bir işe verdiği emeğe yöneliktir. Bu sınıfta ek olarak `employee` üyesi ve onun getter metodu vardır. `Labor Item`, belirli bir çalışan tarafından yapılan işlerin özelleşmiş halidir. İki sınıf arasındaki ilişki kalıtımdır.

5. Duplicate Observed Data (Gözlenen Verinin Çoğullanması)

5.1 Neden GUI sınıfları büyür?

Slayt 34 (ekrandan, 00:20:00):

- Uygulama alanı ile ilgili bir verinin sadece bir GUI sınıfında saklanması doğru değildir.
- Bu veri uygulama alanı/bilgi düzeyi katmanında oluşturulacak yeni bir sınıfa taşınmalı ve
- GUI nesnesi ile yeni bilgi nesnesi arasında Observer tasarım kalıbı kurulmalıdır.

Hoca, bu tarafa eğilim gösterilebildiğini kabul etti. Sonuçta iş mantığıyla ilgili bilgi kullanıcıdan alınacaktır ve bu bilgi grafik kullanıcı arayüzü üzerinden alınır. Arayüzdeki her alan, iş mantığını oluşturacak bir veriyi alabilecek ve değiştirebilecek bileşenlerden oluşur. Bu verinin tutarlılık denetimi de yapılmalıdır ve tutarlılık denetimi aslında iş mantığına ait kurallardır. Bu yüzden ister istemez giriş ya da raporlama ekranlarına iş mantığı kodu yazılır. Bu da her şeyi kullanıcı arayüzü katmanına yığıp ayrı bir iş katmanı olmaması durumuna götürebilir.

Oysa katmanlı tasarımın avantajları daha önce konuşulmuştu: bir katmanla ilgili küçük ya da büyük bir değişiklik söz konusu olduğunda yalnız o katmanda çalışarak istek karşılanabilir ve bu işin diğer katmanları etkilememesi ya da en az düzeyde etkilemesi sağlanabilir.

Hoca düzenlemenin adına da dikkat çekti: "bölün" demiyor, "çoklayın, çoğullayın" diyor. Veri için ayrı bir katman oluşturulur ve ilgili GUI bileşenleriyle yeni bilgi nesneleri arasında Observer kalıbı kurularak birindeki değişikliğin ötekine yansıması sağlanır.

5.2 Interval Window ve Interval örneği

Slayttaki diyagram (ekrandan, 00:20:00):

```
ÖNCE:
  Interval Window
    startField: TextField
    endField: TextField
    lengthField: TextField
    StartField_FocusLost
    EndField_FocusLost
    LengthField_FocusLost
```

```
    calculateLength
    calculateEnd

SONRA:
  Interval Window  - - -|>  <<interface>> Observer
    startField: TextField
    endField: TextField
    lengthField: TextField
    StartField_FocusLost
    EndField_FocusLost
    LengthField_FocusLost
        |
        | 1
        v
  Interval  ----|>  Observable
    start: String
    end: String
    length: String
    calculateLength
    calculateEnd
```

Önceki tasarımda `Interval Window` adlı bir pencere sınıfı vardır. Başlangıç ve bitiş alanlarının yanında işin ne kadar sürdüğünü gösteren bir alan daha vardır. Bu üçünden hangi ikisi verilmişse üçüncüsü hesaplanır: başlangıç ve süre belliyse bitiş (`calculateEnd`), başlangıç ve bitiş belliyse süre (`calculateLength`). Hocanın deyişiyle bunlar iki ayrı iş, ayrı mantıklardır, ama hepsi tek bir sınıfa gömülmüştür.

Düzenleme şunu söyler: bu sınıftaki GUI bileşenleri GUI'de kalsın. Ama madem bu pencere bir aralık (interval) verisini simgelemek ve onunla ilgili veri almak için kullanılıyor, başlı başına bir aralık varlığı da olsun: bir `Interval` sınıfı ve ondan nesneler. Bu sınıfta GUI ile ilgili hiçbir bilgi olmasın. Pencere sınıfında iş mantığının verilerini alabilecek bileşenler ister istemez durur; ama arka katmandaki `Interval` sınıfında GUI'ye dair hiçbir şey yoktur.

Kazanç şudur: GUI değiştirildiğinde, başka bir platforma taşındığında ya da alternatif giriş yöntemlerine geçildiğinde (örneğin veriler bir servisten çekilmeye başlandığında) iş mantığı kısmı değiştirilmeden yalnız veri girişi katmanı değiştirilir.

> **Ek bilgi (derste anlatılmadı):** Java'nın eski standart kütüphanesindeki `java.util.Observer` arayüzü ve `java.util.Observable` sınıfı bu slayttaki adlarla birebir örtüşür; `Observable` alt sınıfı durum değişince `setChanged()` ve `notifyObservers()` çağırır, `Observer` da `update()` ile haber alır. Bu iki tür Java 9'dan beri büyük olasılıkla "deprecated" olarak işaretlidir; yeni kodda genelde kalıbın kendi arayüzleriyle yazılması önerilir.

5.3 Toplam kod uzar, birim küçülür

Hoca bölmenin bir sonucunu sayılarla anlattı. Ortaya, ilk sınıfın eski büyüklüğüne yakın iki sınıf çıkar. Diyelim ki `Interval Window` önceden 1000 satırdı; bölündükten sonra biri 600, öteki 700 satır olabilir, toplam 1300 satır.

Yani 1000 satırlık kod 1300 satıra çıkmıştır; toplamda kod uzamıştır. Ama bakılması gereken toplam değil, birimdir. Her sınıf ayrı bir birimdir. Bir sorun olduğunda hangi sınıf üzerinde çalışılıyorsa onun daha küçük bir parçası üzerinde çalışılır. İki parça da eski halinden kısa olduğu için ($600 < 1000$ ve $700 < 1000$), hocanın deyişiyle "daha kolay yutulabilecek lokmalar" elde edilir. Elde edilen kazanç budur.

6. Divergent Change ve Shotgun Surgery

6.1 Adlar

Hoca bu iki kod kusurunun adlarını biraz esprili bulduğunu, özellikle Shotgun Surgery'nin öyle geldiğini söyledi. "Shotgun", saçma atan çifte tüfektir; Shotgun Surgery "çifteyle ameli-

yat yapmak” gibidir. Slayt 35’te ikisinin ortak Türkçe adı “Değişikliklerin Farklı Noktalarda Yapılmasının Gerekmesi”dir (ekrandan, 00:22:55).

> **Ders sonrası not:** Altyazıda hoca Divergent Change’i “bir değişikliğin çok sık yapılması” diye açıklıyor (00:22:05); bu kısım Whisper’da bozuk da olabilir. “Divergent” sözcüğü “ıraksayan, farklı yönlere giden” anlamındadır; kavramın tanımı aşağıda slayttaki gibidir: bir değişiklik tek sınıfın birçok farklı metoduna yayılır.

6.2 Tanımlar

Gereksinimler değişecek ya da bir hata bulunup düzeltilecek; bu durumlarda koda müdahale edilecektir. Tasarım ne kadar iyiyse bu değişiklikler o kadar az yerde yapılır. Değişiklikler çok yerde yapılıyorsa bu iki kod kusurundan biri var demektir. Slayttaki tanım (ekrandan, 00:22:55):

- Bir gereksinim değişikliğini karşılamak için gereken kod değişiklikleri **aynı sınıfın farklı metotlarında** yapılıyorsa: **Divergent Change**.
- **Birden fazla sınıfta** yapılıyorsa: **Shotgun Surgery**. Hocanın anlatımıyla çok sayıda sınıfın her birinde az sayıda metot değişir.

Örnekler:

- Divergent Change (slayt): “Yeni bir veritabanı ile çalışmak gerekirse şu sınıfın şu 3 metodu değişir.” Hoca sözlü olarak tek bir sınıfın 3 ayrı metodunda toplam 8 değişiklik yapılması örneğini verdi. Bütün değişiklikler tek sınıftadır ama çok sayıdadır.
- Shotgun Surgery (slayt): “Yeni bir tür kredi söz konusu olunca şu 2 sınıftaki şu 5 metot değişmeli.” Hoca sözlü olarak “ya da 3 sınıftaki 8 metot” diye de ekledi. Tek tek sınıflara bakıldığında az sayıda metot değişir, ama sınıf sayısı çok olduğu için toplamda dağınık bir değişiklik resmi çıkar.

6.3 Visitor şeması üzerinde gösterim

Hoca farkı göstermek için önceki derslerde çizilen Visitor örneğinin sınıf şemasını Violet UML Editor’da açtı (ekrandan, 00:25:00). Şemadaki sınıflar:

```
<<Abstract>> ProjectItem
  - name, description : String
  + accept( ProjectVisitor ) {abstract}

Deliverable  (ProjectItem'dan türer)
  - materialsCost, productionCost : double
  + accept( ProjectVisitor )

Task  (ProjectItem'dan türer)
  - projectItems : ArrayList<ProjectItem>
  - timeRequired : double
  + accept( ProjectVisitor )

Project  (ProjectItem'dan türer; ProjectItem'larla bütün-parça ilişkisi)
  - projectItems : ArrayList<ProjectItem>
  + accept( ProjectVisitor )

Contact  (Deliverable ve Task'tan 1 çokluklu ilişki)

<<interface>> ProjectVisitor
  + visitDependentTask(DependentTask)
  + visitDeliverable(Deliverable)
  + visitTask(Task)
  + visitProject(Project)

ProjectCostVisitor, ProjectTimeVisitor  (ProjectVisitor'ı gerçekler)
Report
```

```
  + Report()
  + main()
```

Şemadaki notta “Önceki çözümle aynı olan üyeler gösterilmemiştir” yazar.

Shotgun Surgery için hoca kırmızı kalemle dağınık noktaları işaretledi: `ProjectItem`’ın `accept` metodu, `Deliverable`’daki bir alanın türü, `Project`’in `accept` metodu, `ProjectVisitor`’daki bir metot, `Report` ve `Contact`. Beş-altı ayrı değişiklik, şemanın her yerine dağılmıştır.

Divergent Change için ise yeşil kalemle `ProjectVisitor`’ın `visitDependentTask`, `visitTask` ve `visitProject` metotlarını ve `ProjectCostVisitor`’daki iki değişikliği işaretledi. Yeşiller de çok sayıdadır, ama daha toplu bir alandadır. Kırmızılar ise dağınıktır; hocanın deyişiyle “çifteyle ateş etmişim gibi”.

6.4 Çözüm

Slayttaki çözüme götüren düzenlemeler (ekrandan, 00:22:55):

- **Extract Class:** Söz konusu metotlar ve ilgili üyelerden yeni bir sınıf çıkarılır. Hoca “bir ya da iki” yeni sınıf çıkarılabileceğini söyledi.
- **Move Method / Move Field:** Söz konusu üyeler daha uygun bir mevcut sınıfa taşınır. Hoca bunun özellikle Shotgun Surgery’de söz konusu olduğunu söyledi. Metotlarda değişiklik yapılıyorsa Move Method, alanlarda yapılıyorsa Move Field, ikisinde birden yapılıyorsa ikisi birlikte kullanılır.

Böylece sık değiştirilen ya da yaygın biçimde değiştirilen alanlar ve metotlar bir ya da iki sınıfta toplanır. Mevcut sınıflarla bu yeni sınıf arasında hangisi işe yarıyorsa o tür bir ilişki kurulur: sahiplik, parça-bütün ya da kullanma.

7. Move Method (Metot Taşınması)

7.1 Tanım ve ölçüt

Slayt 36 (ekrandan, 00:30:00):

- Bir metot tanımlandığı sınıf içinden çok, başka bir sınıftan çağrılıyorsa; en çok hangi sınıftan çağrılıyorsa o sınıfa taşınmalıdır.
- Bu metot ya eski sınıfından tamamen silinir, ya da eski sınıftaki hali sadece yeni sınıftaki halini çağıracak şekilde değiştirilir (delegation).
- Bu düzenleme pek çok kokuyu giderdiğinden çok sık kullanılır ve bazı düzenlemelerin alt adımıdır (örneğin Extract Class).

Hoca, bir metodun tanımlandığı sınıftan çok başka sınıflardan çağrıldığı görülünce “galiba bu metodu doğru yere koymamışız” fikrinin akla gelmesi gerektiğini söyledi. Metot en çok hangi sınıftan çağrılıyorsa oraya taşınır. Metodun kullandığı alanlar varsa onların da taşınması gerekebilir.

Slayttaki örnek:

```
ÖNCE:                  SONRA:
  Class 1                Class 1
    aMethod()              (boş)
  Class 2                Class 2
    (boş)                  aMethod()
```

Hocanın sayısal örneği: `aMethod` `Class 1`’de tanımlıdır, ama `Class 2`’nin içindeki 8 ayrı metot bu metodu çağırıyorsa, bu metot baştan `Class 2`’ye konmalıymış demektir ve düzenleme yapılmalıdır.

7.2 Eski sınıfa ne olacak?

Taşınan metot eski sınıftan tamamen silinebilir. Ancak bu, sınıfın dışa açtığı genel metot arayüzünü (API) değiştirir. Hoca, bu yüzden refactoring kapsamında silmenin tercih edilmeyebileceğini söyledi: refactoring’in en önemli hedeflerinden biri geriye doğru uyumluluğu korumak,

yani dışarıya verilen hizmet arayüzünü değiştirmeden iç yapıyı değiştirmektir. Metodu tamamen silmek refactoring'in özüne biraz aykırıdır.

Daha iyisi, metodu eski yerinden silmemektir. Metodun imzası kalır; gövdesi ise eski yere gelen çağrıyı metodun taşındığı yeni yere iletir. Buna delegation (çağrı iletme) denir.

> **Hocanın vurgusu:** Refactoring'in en önemli hedeflerinden biri geriye doğru uyumluluktur; dışa açılan arayüz değişmeden iç yapı değişir. Bu yüzden taşınan metot eski yerde silinmek yerine çağrı ileten bir metot olarak bırakılabilir.

7.3 Ayrı bir düzenleme mi?

Move Method başka düzenlemelerin alt adımı olarak da düşünülebilir. Örneğin Extract Class yapılırken eski sınıftan bazı üyeler ve metotlar yeni sınıfa taşınır. Hoca, bu yüzden başlı başına bir düzenleme sayılmayabileceğini, ama kendi görüşüne göre ayrı bir düzenleme olmayı hak ettiğini söyledi: bir sınıf çıkarmak zorunda kalmadan, metot mevcut sınıflar arasında da taşınabilir.

8. Primitive Obsession (İlkel Tiplerin Aşırı Kullanımı)

Hoca "obsession" sözcüğünün "takıntı" demek olduğunu, yani kişinin kendini kurtaramadığı, ısrarla yaptığı hareket ya da düşünceleri anlattığını söyledi. Burada ilkel tiplerin kullanımına takıntıdan söz edilir; ama hoca kendisi bunu "aşırı kullanım" diye çevirmeyi tercih etti.

Slayt 37 (ekrandan, 00:30:25):

- Yapısal programlamadan gelenler ilkelleri aşırı yoğun olarak kullanır.
- Bu yoğun kullanımın farklı biçimlerini kelimelerle tarif etmektense her birine yönelik yeni düzenlemeleri incelemek tercih edilmiştir.
- Çözüme götürecek düzenlemeler: Replace Data Value with Object, Replace Type Code with Class, Replace Type Code with Subclasses, Replace Type Code with State/Strategy, Replace Array with Object.
- Extract Class ve Introduce Parameter Object düzenlemeleri de daha nesneye yönelimli bir program hazırlamak için kullanılabilir.

Hoca, yapısal programlamada uzun süre çalışmış kişilerin ilkel tipleri yoğun kullanma eğiliminde olduğunu söyledi. Bunun farklı biçimleri olduğu için hepsini burada uzun uzun tartışmak yerine her birini ilgili aşırı kullanım türüyle birlikte konuşmanın daha doğru olacağını belirtti. Hepsi "Replace X with Y" kalıbında adlandırılmış düzenlemelerdir.

Listenin sonundaki iki düzenleme daha önce görülmüştü: Extract Class bu derste, Introduce Parameter Object ise önceki derste, özellikle Long Method ve aşırı uzun metot parametre listelerini kısaltmak için. Hoca bunların da tasarımın üzerindeki "yapısallık kokusunu" atmak ve daha nesneye yönelimli kokmasını sağlamak için kullanılabileceğini hatırlattı.

9. Replace Data Value with Object

9.1 Tanım ve örnek

Slayt 38: "Bir üye alan, hele ki bu alan ile ilgili çeşitli davranışlar söz konusu ise, bir sınıf ile simgelenmelidir." (ekrandan, 00:32:35).

```
ÖNCE:
  Order
    customer: String

SONRA:
  Customer  <>---------- 1 -->  Order
    name: String
```

Slaytta "SONRA" kısmında `Customer` tarafında içi dolu elmas (composition) ve `Order` ucunda 1 çokluğu görünür (ekrandan, 00:32:35).

9.2 String olarak müşteri neden kötü?

Önceki tasarımda siparişi kimin verdiği bilgisi `Order` sınıfında `customer` adlı `String` türünden bir alanda tutulur. Örneğin bir müşteri bir kalemden yüz paket sipariş vermiştir ve sipariş kaydında müşterinin adı bir metin olarak saklanır.

Siparişi veren kişinin başka bilgilerine ulaşmak gereken senaryolarda sorun çıkar. Hocanın örneği: en çok sipariş veren ilk 100 müşteri kimdir ve bunların iletişim bilgileri ile diğer bilgileri nelerdir, gibi bir rapor. Önceki tasarımda bunun için sistemdeki bütün müşteri kayıtları taranıp adı siparişteki metinle aynı olan müşteri bulunmalıdır. Bu çok verimsizdir.

Hoca bunu veri tabanı odaklı tasarımın (database-driven design) bir kalıntısı olarak niteledi: ad alanı bir tür yabancı anahtar (foreign key) gibi kullanılmaktadır. Veri tabanında bu iş verimli yapılabilir, ama nesnelerde hiç verimli değildir: bir listede kişiler, bir listede siparişler tutulur ve bütün kişi nesneleri tek tek dolaşılıp "senin adın bu mu?" diye sorulur. Aynı durum bir tam sayı kimlik (ID) alanı için de geçerlidir. Başlı başına bir varlık olması, yani bir sınıftan türetilen nesneler olması gereken üye alanlar ilkel tiplerle tutulmamalıdır.

Doğru olan, bir müşteri sınıfı ve bir sipariş sınıfı olması ve aralarında ilişki kurulmasıdır. Böylece bir `Order` nesnesinin müşterisi alındığında tek adımda bütün bir nesne elde edilir ve bu müşteri nesnesi başka bütün işler için rahatça sorgulanabilir.

Slaytın alt kısmı (ekrandan, 00:32:35):

- Programın tümü incelendiğinde, belli bir müşterinin tüm siparişlerinin aranması gibi işlemler yapıldığı görülecektir.
- Bu işlemler çoğaldıkça, Müşteri sınıfı ayrılmadığı sürece Sipariş sınıfı aşırı büyüyecektir.
- İyisi mi şimdiden bu sınıfları ayıralım.

Hoca bu işlerin zaten çoğalma eğiliminde olduğunu, müşteri sınıfı siparişten ayrılmadıkça sipariş sınıfına yazılan kodun aşırı şişeceğini söyledi. Bu yüzden erkenden, şimdiden ayırmak gerekir.

10. Replace Type Code with Class

10.1 Tanım ve örnek

Slayt 39: "Yapısal programlamada sabit değişkenlerin kullanılması alışkanlığı ile ilgilidir." (ekrandan, 00:36:40).

```
ÖNCE:
  Person
    O : int          (statik)
    A : int          (statik)
    B : int          (statik)
    AB : int         (statik)
    bloodGroup : int

SONRA:
  Person
     | 1
     v
  BloodGroup
    O : BloodGroup   (statik)
    A : BloodGroup   (statik)
    B : BloodGroup   (statik)
    AB : BloodGroup  (statik)
```

Slayttaki `O`, `A`, `B`, `AB` alanlarının altı çizilidir; UML'de altı çizili üye statik demektir.

10.2 Sorun: anlamsız değerler

İlkellerin farklı bir kullanım biçimi, sabit değişkenlere fazla alışkanlıktır. Hocanın örneği: kişilerin kan grupları tutulmalıdır. Dört kan grubu vardır: 0, A, B ve AB. Bu yüzden `Person` sınıfında

dört ayrı sabit tanımlanır. Hoca bunların hem `final` hem `static` olduğunu, ikisi birlikte olduğu için `public` de tanımlanabileceğini söyledi. Kişinin kan grubunu belirtmek için ise normal bir üye alan vardır: `bloodGroup`.

`bloodGroup`'a `O` atanmışsa 0 grubu, `A` atanmışsa A grubu anlaşılır. Arka planda bu sabitlerin içinde tam sayı değerler vardır; örneğin 0 grubu için 0, A grubu için 1, B grubu için 2, AB grubu için 3 atanmış olabilir.

Hocanın sorusu: `bloodGroup` `int` olduğuna göre bu alana 8 değeri atanırsa ne olacak? Öyle bir kan grubu yoktur, değerin bir anlamı yoktur. Böyle ters durumlara düşme olasılığı yüksektir.

10.3 Çözüm ve enum

Çözüm, kan grubu için ayrı bir sınıf oluşturmak, her grubu bu sınıfın kendi türünden (`BloodGroup`) statik bir alanı olarak tanımlamak ve `Person` ile `BloodGroup` arasında birebir sahiplik ilişkisi kurmaktır. Böylece `bloodGroup` alanına ancak bu dört nesneden biri atanabilir.

Hoca, birçok dilde bunun altyapısının hazır olarak sunulduğu `enum` yapısının bulunduğunu ve onun da kullanılabileceğini söyledi (slayt: "Ya da kısaca enum kullanabilirsiniz"). Genel ilke, farklı bilgi değerlerini simgelemek için farklı sabit değerler kullanmak yerine bunları başlı başına bir tür olarak düşünmektir. Örneğin kan grubu için ayrı bir tür, Rh faktörü (pozitif ya da negatif) için ayrı bir tür; müşteriler için normal, düzenli, platin gibi kategoriler varsa onlar için ayrı bir enum. Her biri ilkel sayısal değişkenler yerine kendi enum türünden ayrı bir üye alan olarak kullanılmalıdır.

Slaytta "Enum'u bilmeyen yoksa sonraki 2 yansıyı atlayabiliriz" yazar (ekrandan, 00:38:20). Hoca enum'u bilmeyen olmadığını düşündüğü için enum kullanımını gösteren iki slaytı atladı.

> **Ek bilgi (derste anlatılmadı):** Java'da bu örnek `enum BloodGroup { O, A, B, AB }` biçiminde yazılabilir; `Person` sınıfında `private BloodGroup bloodGroup;` alanına derleyici yalnız bu dört değerden birini (ya da `null`) atamaya izin verir. Slayttaki elle yazılmış "statik alanları kendi türünden olan sınıf" çözümü, büyük olasılıkla dile enum eklenmeden önceki "typesafe enum" kalıbıdır.

11. Replace Type Code with Subclasses

11.1 Tanım ve sınıf diyagramı

Slayt 42: "Bir sınıfın davranışını belirleyen bir ilkel tip kodu yerine kalıtımın kullanımı daha uygun olacaktır." (ekrandan, 00:39:45).

```
ÖNCE:
  Employee
    ENGINEER : int   (statik)
    SALESMAN : int   (statik)
    type : int

SONRA:
          Employee
             ^
     +------+------+
  Engineer      Salesman
```

11.2 Sorun: her metotta switch

Bir sınıfın nesnelerinin farklı alt türleri olabilecekse bunlar bir tip kodu altında toplanabilir. Hocanın örneği bir `Employee` sınıfıdır. Şirkette mühendisler, satış elemanları, hizmet elemanları, çağrı merkezi elemanları gibi farklı kategorilerde çalışanlar olabilir. Her biri sabit değerler olarak tanımlanır ve çalışanın hangi türden olduğu bir üye alanla belirlenir.

Böyle bir sınıfın kaynak koduna bakıldığında şu görülür: çalışan bir iş, bir görev yapacaktır; maaşı, bazı ödemeleri, ikramiyesi hesaplanacaktır. Bu metotların her birinde ya bir `switch-case` yapısı ya da uzun bir `if - else if` zinciri olur: "`type == ENGINEER` ise şöyle yap, `else if`

`type == SERVICEDESK` ise böyle yap". Hoca bunun düzenleme gerektiğinin bir göstergesi olduğunu ve daha önce görülen Long Method ile Large Class kusurlarına da yol açtığını söyledi.

11.3 Kod örneği

Slayt 43'teki kod (ekrandan, 00:42:40):

```
/*** ÖNCE: ***/
class Employee {
  private int _type;
  static final int ENGINEER = 0;
  static final int SALESMAN = 1;
  static final int MANAGER = 2;
  Employee (int type) {
    _type = type; }
  public void work( ) {
    switch(_type) {
      case ENGINEER:
        //do something
        break;
      case SALESMAN:
        //do something
        break;
      case MANAGER:
        //do something
        break;
    }
  }
}
```

```
/*** SONRA: (slayttaki hali) ***/
abstract class Employee {
  public abstract void work(){};
}
public class Engineer extends
  Employe {
  public void work( ) {
    //do something as an engineer
  }
}
public class Salesman...
```

Slayttaki notlar:

- Employee'nin önceki halinde case default varsa, sonraki halde Employee sınıfı abstract yapılmaz.
- Bu örnek Replace Conditional with Polymorphism düzenlemesine de benzemektedir.

  Ders sonrası not: Slayttaki "SONRA" kodu Java'da derlenmez. `public abstract void work(){};` satırı "abstract methods cannot have a body" hatası verir; doğrusu gövdesiz `public abstract void work();` biçimidir. `extends Employ` da `extends Employee` olmalıdır. Düzeltilmiş hali (`abstract class Employee { public abstract void work(); }` ve `class Engineer extends Employee { public void work() { ... } }`) ajan tarafından `java` ile çalıştırılarak denendi; slayttaki hali aynı hatayla reddedildi.

11.4 Anlatım

Farklı çalışan türleri ayrı sınıflar yapılır ve `Employee`'nin alt sınıfları olur. `Employee`'nin her metodundaki `if-else` ya da `switch` içindeki her `case`, ilgili alt sınıfa taşınarak dağıtılır. Örnekte `work` metodundaki `case ENGINEER` gövdesi `Engineer` sınıfının `work` metoduna, `case SALESMAN` gövdesi `Salesman` sınıfının `work` metoduna gider.

Sonuçta `Employee` sınıfı çok daha derli toplu bir sınıf olur. Hangi tür çalışanla ilgili iş söz konusuysa o iş yalnız o çalışan türünün sınıfında bulunur, başka yerde olmaz. Örneğin mühendisin belli bir konudaki çalışma biçimi değişecekse `Salesman` ve diğer sınıflara dokunmadan bu iş yapılabilir.

Rastgele, türü belirsiz bir çalışan nesnesine gerek yoksa `Employee` sınıfı soyut (abstract) tanımlanabilir. Ama `switch` içinde bir `default` durumu varsa, o varsayılan çalışma biçimi `Employee` sınıfında kalabilir. Bu durumda o biçimi yürütecek nesneler oluşturabilmek için `Employee` soyut yapılmaz.

Hoca bu düzenlemenin Replace Conditional with Polymorphism'e benzediğini, ama onun daha özelleşmiş hali olduğunu söyledi. Burada değiştirilen herhangi bir sıradan `switch` ya da `if-else` değil, işi yapacak nesnenin türüne göre farklı davranışlar yürüten bir tip kodudur. Fowler bu yüzden düzenlemeyi "Replace Type Code with Subclasses" diye adlandırmıştır.

12. Replace Type Code with State/Strategy

12.1 Ne zaman?

Slayt 44: "Önceki örnekte kalıtım kullanımı bir nedenle mümkün değilse, State veya Strategy tasarım kalıbı da kullanılabilir." (ekrandan, 00:45:10).

Önceki çözümde bir kalıtım zinciri kurulmuştu. Hoca, alt sınıf oluşturmak istenmeyebileceğini söyledi: oluşturulması düşünülen sınıfın başka sınıflardan kalıtım alması gerekebilir, hatta o başka sınıftan kalıtım almanın çok daha önemli olduğu fark edilebilir. Bu durumda kusuru düzeltmek için State ya da Strategy kalıbı uygulanır ve `Employee` hiyerarşisinde kalıtım kullanılmaz.

12.2 Diyagram

```
ÖNCE:
  Employee
    ENGINEER : int   (statik)
    SALESMAN : int   (statik)
    type : int

SONRA:
  Employee  ---- 1 ---->  Employee Type
                                ^
                          +------+------+
                      Engineer      Salesman
```

(ekrandan, 00:45:10)

Önceden `Employee`'nin içinde mühendis, satış elemanı, yönetici gibi türler sabitlerle tutuluyordu. Artık `Employee`'nin `Employee Type` türünden bir üye alanı vardır ve bu türden bir nesneyle birebir sahiplik ilişkisi içindedir. Bu nesne somut olarak `Engineer`, `Salesman`, `Manager` her neyse odur. `Employee`'nin `work` metodu içinde bu `Employee Type` nesnesinin `work` metodu çağrılır. Çalışan hangi tür olarak belirlenmişse onun tür nesnesi atanmış olduğundan, o somut alt sınıfın `work` metodu nasıl kodlanmışsa o çalışır. Hoca, Strategy kalıbı bilindiği için bunun uzun anlatılmasına gerek olmadığını da ekledi.

12.3 Daha esnek: rol değişimi

Hoca bu çözümün öncekinden daha esnek olduğunu bir senaryoyla anlattı. Bir çalışan önce mühendis olarak işe alınmıştır. Beş altı yıl çalışıp yorulmuş, artık başka türde işler yapmak istemiştir; amiri onu satış departmanına atayabilir.

- Kalıtım çözümünde bir `Engineer` nesnesini doğrudan bir `Salesman` nesnesine dönüştürmek mümkün değildir. Yeni bir `Salesman` nesnesi oluşturulur, mevcut `Engineer` nesnesinin durum bilgisi `Salesman`'a aktarılır, `Salesman`'a özgü durum bilgileri varsa onlar ayrıca alınır, sonra `Engineer` nesnesi silinip `Salesman` nesnesi tutulur. Bu epey ayrıntılı bir iştir.

- Strategy çözümünde ise önce mühendis rolü verilmiş olan `Employee` nesnesine artık satış elemanı rolü verildiği söylenir. Bu iş tek bir atamayla yapılır ve `Employee` nesnesinin silinmesi gerekmez.

  Hocanın vurgusu: Strategy çözümü kalıtım çözümünden daha esnektir; rol değişimi tek bir atamayla yapılır ve nesneyi silip yeniden oluşturmak gerekmez.

13. Replace Array with Object

13.1 Tanım

Slayt 45: "Çok boyutlu dizilerde kayıt tutmak yerine, tek bir nesne kullanın." (ekrandan, 00:50:00).

Kayıtlar çok boyutlu dizilerde ya da çok sayıda tek boyutlu dizide tutulabilir. Hoca bu alışkanlığın kaynağını şöyle anlattı: lisansta birinci sınıf öğrencileri henüz `struct` gibi yapıları görmemişken, bir varlıkla ilgili farklı türden verileri tutmanın bir yolu olarak bu yöntem gösterilir. Bazıları bunu bir kez görünce bütün hayatları boyunca o şekilde kod yazmaya devam edebilir. Oysa nesneye dayalı yaklaşımda bunun çok daha iyisi öğrenilmiştir.

13.2 Örnek

Örnekte takımların adları, puanları ve attığı gol sayıları tutulmaktadır. Slayttaki kod (ekrandan, 00:50:00):

```
/*** ÖNCE: ***/
String[][] scores = new
   String[3][teamCount];
scores[0][0] = "Liverpool";
scores[1][0] = "15";
scores[2][0] = "67";

/*** SONRA: (slayttaki hali) ***/
Score[teamCount] scores;
scores[0] = new
   Score("Liverpool");
scores[0].setWins("15");
scores[0].setPoints("67");
```

Hoca üç aşamayı anlattı:

1. **İki boyutlu `String` dizisi:** Aynı takımın farklı verileri aynı dizinin farklı hücrelerindedir. Burada sayısal değerler bile string olarak tutulmak zorundadır, çünkü dizi `String` türündendir.
2. **Paralel tek boyutlu diziler (sözlü):** Belki biraz daha iyisi `teamNames` (`String` dizisi), `points` (`int` dizisi) ve `average` (`int` dizisi) gibi üç ayrı tek boyutlu dizidir. Her dizinin aynı indisindeki veriler aynı takıma aittir. Ama burada da gereksiz bir dağılma vardır: aynı varlığın verileri farklı yerlerdedir, birini güncellemeyi unutmak gibi birçok şey ters gidebilir.
3. **Nesne dizisi:** `Score` diye bir sınıf oluşturulur. Hoca bunun adının belki `Team` de olabileceğini, ama odak puanlar olduğu için `Score` dediğini söyledi. `Score` sınıfının üç üyesi olur: takım adı (`String`), puan ve üçüncü bir sayısal değer. Sonra bu sınıf türünden nesnelerin dizisi kullanılır.

   Ders sonrası not: Hoca iki boyutlu dizide "her satır bir takım; ilk sütun ad, ikinci sütun puan, üçüncü sütun averaj" dedi. Slayttaki kodda ise `new String[3][teamCount]` ile ilk indis veri türünü, ikinci indis takımı gösterir (`scores[0][0]` ad, `scores[1][0]` ve `scores[2][0]` aynı takımın sayısal değerleri). Yani slaytta her **sütun** bir takımdır. Fikir değişmez; yalnız yön farklıdır. Ayrıca hoca sayısal alanları sözlü olarak "puan" ve "averaj" diye andı, slaytta ise `setWins` ve `setPoints` vardır.

   Ders sonrası not: Slayttaki `Score[teamCount] scores;` bildirimi Java'da geçerli değildir (derleyici "not a statement" hatası verir). Doğrusu `Score[] scores = new`

`Score[teamCount];` biçimidir. Ajan, düzeltilmiş bildirimi `Score("Liverpool")`, `setWins("15")` ve `setPoints("67")` çağrılarıyla birlikte `java` ile çalıştırarak denedi; çıktı `Liverpool 15 67` oldu. Slayt sayısal değerleri hâlâ string olarak geçirir; nesneye geçişin bir kazancı da bu alanları `int` yapabilmektir.

14. Feature Envy (Özellik Kıskançlığı)

14.1 Tanım

Hoca "Feature Envy"yi "özellik kıskançlığı" diye çevirdi. Slayt (ekrandan, 00:50:50):

- Bir metot ait olduğu sınıfın üyelerinden çok başka sınıfların üyelerine erişiyorsa bu duruma "özellik kıskançlığı" adı verilir.
- "Çok" sıfatı ile o koddaki ortalama (veya %20 gibi bir eşikten fazla) farklı sınıf üyesi erişim sayısının üzerinde bir sayı ifade edilmektedir.
- Sorumlulukların doğru atanmadığının göstergesi olabilir.
- Elbette bağlaşım ilkesi nedeniyle başka sınıflardan nesnelere erişmek gerekecektir; bu nedenle hiçbir sınıf sadece kendi metotlarını kullanmaz. Öyle olsaydı bu kez de uyum ilkesini olumsuz yönde etkilemiş olurduk.
- Strategy ve Visitor kalıpları bir özellik kıskançlığı durumu olarak yorumlanmamalıdır.
- Çözüm için olası refactoring eylemleri: Move Method; Extract Method (kıskançlık metodun sadece bir kısmındaysa o kısım çıkarılıp kıskanılan üyelerin bulunduğu sınıfa taşınabilir).

14.2 Move Method'la ilişkisi

Hoca bu kusurun Move Method'a benzediğini, çözümlerinden birinin de zaten Move Method olduğunu söyledi. Ama bakış farklıdır. Move Method'da belirli bir üyenin başka sınıflardan daha çok kullanılması söz konusuydu. Feature Envy olaya tersten bakar: bir metot, kendi sınıfındaki üyelerden çok başka sınıflardaki üyelere erişir.

14.3 "Çok" ne demek?

"Çok" demek için bir eşik vermek gerekir. Hoca bunun birkaç yolunu saydı. Projenin bütününe bakıp ortalama erişim sayısı hesaplanabilir. Ya da makul bir eşik kişinin kendisi tarafından belirlenebilir. Bunun için daha önce yapılmış ve başarılı sayılan projelerde bu ortalamalar hesaplanıp, örneğin bunun en fazla %10 üstüne kadar hata payı tanınabilir; daha fazlasında kod kusuru var denir. Slaytta ise %20 gibi bir eşik verilmiştir. Bu sıkıntının kök nedeni genelde o üyeyle ilgili sorumlulukların doğru atanmamış olmasıdır.

Hoca, bağlaşımın sıfır olamayacağını da vurguladı. Bir nesne bir sorumluluğu yerine getirirken başka nesnelerle ilişki içinde olacaktır; bu kaçınılmazdır. Hiçbir sınıf yalnız kendi metotlarını kullanmaz. Ama oransal olarak bakıldığında başka sınıflara erişim %20 gibi bir eşikten fazla da olmamalıdır.

14.4 Strategy ve Visitor istisnası

Bu tanım akla Strategy ve Visitor kalıplarını getirebilir. Her Strategy gerçeklemesi, işini yapabilmek için atanmış olduğu nesnenin durum bilgisini sorgulamak zorunda kalabilir. Hoca bunun özellik kıskançlığı olmadığını söyledi: zaten o işin bu yeni sınıfın nesnesine yaptırılması istenmiştir, dolayısıyla yeni sınıf ihtiyaç duyduğu bilgileri mecburen alacaktır.

Visitor'da da durum aynıdır. Bir visitor nesnesi ağaç yapısındaki düğümleri tek tek ziyaret eder; düğümler visitor'ı kabul eder (accept) ve kendilerini ona verir; visitor da ziyaret ettiği nesnenin durumunu sorgulayarak yapması gerekeni yapar. Bu nesnelerin böyle kullanılması zaten istenmiştir ve kalıbın gereklilikleri bunu gerektirir. Bu yüzden bunlar Feature Envy değildir.

> **Hocanın vurgusu:** Strategy ve Visitor'da başka nesnenin durumunu sorgulamak Feature Envy sayılmaz; o işin yeni sınıfa yaptırılması zaten bilinçli bir tasarım kararıdır.

14.5 Çözüm

İlk akla gelen çözüm Move Method'dur: farklı yerleri kıskanan metot, en çok kıskandığı yer neresiyse oraya taşınır. Böylece daha az kıskanç hale gelir. Hoca, hiç kıskanç olmayan bir metot diye bir şey olmadığını, ama kıskançlığın mümkün olduğu kadar en aza indirilebileceğini söyledi.

Kıskançlık taşınması düşünülen metodun tümünde değil de yalnız bir kısmındaysa, metot kökten taşınmaz. Yalnız o kısım Extract Method ile çıkarılır ve kıskandığı yere konur. Move Method ile Extract Method seçeneği arasındaki fark budur.

15. Kapanış: Kalıplar ve Kod Kusurları

Hoca, bu dersle hedeflenen kod kusuru ve refactoring içeriğinin tamamlandığını söyledi. Fowler'ın kitabında bundan çok daha fazla kod kusuru ve refactoring eylemi vardır. Hoca bunların içinden bir seçme yaptığını anlattı: hangi kod kusurunun hangi refactoring eylemleriyle birlikte kullanılabildiğine bakmış ve kendi içinde en yoğun ilişkiye sahip bir alt kümeyi seçmiştir. Hepsini anlatmaya kalkılsa her hafta üçer saat dolu dolu çalışılsa bile yetişmeyeceğini söyledi.

Slaytın son kısmı ("Bir Tasarımın Ardışıl Düzenlemeler ile Değiştirilmesi"): şimdiye kadar incelenen düzenlemelerden bazılarının birlikte kullanılarak yapısal programlama yaklaşımı izleri taşıyan bir tasarımın nasıl değiştirildiğine bir örnek için Fowler'ın Refactoring kitabının 1. bölümü incelenebilir (ekrandan, 00:50:50).

Hoca bu iki konuyu iyi tasarımın iki yüzü olarak bağladı. Tasarım kalıplarıyla bir şeyin nasıl doğru yapıldığı öğrenilir. Kod kusurları üzerinde çalışarak bir şeyin nerede yanlış olduğu tespit edilir ve nasıl düzeltileceği öğrenilir. Bu ikisi iyi bir tasarım yapabilmenin, bunu öğrenmenin ve bu konuda deneyim kazanmanın iki ayrı yüzüdür; hocanın benzetmesiyle "elmanın iki yarısı, paranın iki yüzü"dür ve birlikte bilinmelidir.

> **Hocanın vurgusu:** Tasarım kalıpları ile kod kusurları iyi tasarımın iki yüzüdür; ikisi birlikte bilinmelidir.

Sonraki adım olarak hoca, Fowler'ın kitabında en başta yaptığı bütüncül örneği bu derste en sona bırakacaklarını söyledi: önce bütün kod kusurları ve düzeltme yöntemleri çalışıldıktan sonra, epey kötü kokan kusurlu bir kod adım adım düzenlemelerle iyileştirilecektir.

> **Ek bilgi (derste anlatılmadı):** Fowler'ın "Refactoring" kitabının 1. bölümündeki bu bütüncül örnek, birinci baskıda bir video kiralama dükkânının müşteri ekstresini üreten `statement()` metodudur. Örnekte Extract Method, Move Method, Replace Temp with Query ve Replace Type Code with State/Strategy gibi bu derste ve önceki derste görülen düzenlemeler sırayla uygulanır; büyük olasılıkla hocanın sonraki derste yapacağı örnek budur.

Örnek Sorular

Bu derste ekranda soru ya da alıştırma gösterilmedi. Aşağıdaki sorular hocanın ders sırasında sözlü olarak sorduğu ve kendisinin yanıtladığı sorulardır.

Soru 1 (derste sözlü, 00:37:21): `Person` sınıfında kan grupları `static final int O = 0, A = 1, B = 2, AB = 3` sabitleriyle, kişinin kan grubu da `int bloodGroup` alanıyla tutuluyor. `bloodGroup` alanına 8 değeri atanırsa ne olur? Bu tasarım nasıl düzeltilir?

> **Çözüm (derste verildi):** Derleyici buna izin verir, çünkü alan `int`'tir; ama 8 diye bir kan grubu yoktur, değer anlamsızdır ve program tutarsız bir duruma düşer. Düzeltme Replace Type Code with Class'tır: ayrı bir `BloodGroup` sınıfı oluşturulur, `O`, `A`, `B`, `AB` bu sınıfın kendi türünden statik alanları olur ve `Person` bir `BloodGroup` nesnesine birebir sahip olur. Kısaca `enum` da kullanılabilir.

Soru 2 (derste sözlü, 00:27:53): Move Method ile bir metot başka bir sınıfa taşındığında eski sınıftaki metoda ne olmalıdır?

> **Çözüm (derste verildi):** Tamamen silinebilir, ama bu sınıfın dışa açtığı API'yi değiştirir ve refactoring'in geriye uyumluluk hedefine aykırı düşer. Tercih edilen yol,

metodun imzasını eski yerde bırakmak ve gövdesinde çağrıyı yeni yere iletmektir (delegation).

Soru 3 (derste sözlü, 00:45:55 - 00:47:00): Bir çalışan mühendislikten satış departmanına geçirilecek. Replace Type Code with Subclasses çözümünde ve Replace Type Code with State/Strategy çözümünde bu nasıl yapılır? Hangisi daha esnektir?

> **Çözüm (derste verildi):** Alt sınıf çözümünde `Engineer` nesnesi doğrudan `Salesman` olamaz: yeni bir `Salesman` nesnesi oluşturulur, durum bilgisi aktarılır, satışa özgü bilgiler eklenir, eski `Engineer` nesnesi silinir. Strategy çözümünde `Employee` nesnesinin `Employee Type` üyesine yeni bir `Salesman` nesnesi atanır; tek bir atama yeterlidir ve `Employee` nesnesi silinmez. Strategy çözümü daha esnektir.

Soru 4 (derste sözlü, 00:53:09): Strategy ya da Visitor kalıbında bir nesne başka bir nesnenin durumunu yoğun biçimde sorguluyor. Bu Feature Envy sayılır mı?

> **Çözüm (derste verildi):** Hayır. Bu kalıplarda işin ayrı bir sınıfın nesnesine yaptırılması zaten istenmiştir; o nesnenin gerekli bilgiyi almak için asıl nesneyi sorgulaması kaçınılmazdır ve kalıbın gereğidir.

Soru 5 (derste sözlü, 00:20:26 - 00:21:12): 1000 satırlık `Interval Window` sınıfı Duplicate Observed Data ile 600 ve 700 satırlık iki sınıfa bölündü. Toplam kod 1300 satıra çıktığına göre bu düzenleme kodu kötüleştirmiş midir?

> **Çözüm (derste verildi):** Hayır. Toplam 300 satır artmıştır, ama ölçüt toplam değil birimdir. Her iki sınıf da eski halinden (1000 satır) kısadır; bakım sırasında yalnız ilgili sınıf üzerinde çalışılır, yani daha küçük ve kolay anlaşılır parçalarla uğraşılır.

Dersin Toparlaması

Bu derste kod kusurları dizisi tamamlandı. Önce Large Class ele alındı: kök nedeni sınıfa çok sayıda ve birbiriyle ilgisiz sorumluluk atanmasıdır, sonucu uyumun düşmesidir. Çözüm olarak Extract Class (ilgisiz ikinci işi yeni sınıfa taşımak), Extract Subclass (her zaman kullanılmayan üyeleri alt sınıfa taşımak) ve Duplicate Observed Data (uygulama verisini GUI'den ayırıp Observer ile eşgüdümlemek) görüldü. Ardından değişikliklerin dağınıklığıyla ilgili iki kusur karşılaştırıldı: Divergent Change'de değişiklikler tek sınıfın birçok metodunda, Shotgun Surgery'de birçok sınıfa saçılmış haldedir; çözüm Extract Class ile Move Method/Move Field'dır. Move Method'da metodun en çok çağrıldığı sınıfa taşınması ve eski yerde çağrı iletilerek geriye uyumluluğun korunması vurgulandı.

Dersin ikinci yarısı Primitive Obsession'a ayrıldı: ilkel tipler yerine sınıf (Replace Data Value with Object), tip kodu yerine sınıf ya da enum (Replace Type Code with Class), davranışı belirleyen tip kodu yerine kalıtım (Replace Type Code with Subclasses) ya da kalıtım uygun değilse State/Strategy, paralel ya da çok boyutlu diziler yerine nesne dizisi (Replace Array with Object). Son olarak Feature Envy ve onun Move Method/Extract Method ile çözümü işlendi; Strategy ve Visitor kalıplarının bu kusur sayılmadığı belirtildi. Hoca, tasarım kalıpları ile kod kusurlarının iyi tasarımın iki yüzü olduğunu söyleyerek konuyu kapattı. Sonraki ders için akademik bağlantı, Fowler'ın Refactoring kitabının 1. bölümündeki bütüncül örneğin bu düzenlemelerle adım adım yapılmasıdır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.